

PROJETO DE FIM DE SEMANA · QUARKUS

MotoLog

API REST de manutenção de motos — controle de serviços, horas de motor e custos, com autenticação JWT e redefinição de senha.

Quarkus 3.x

Hibernate Panache

PostgreSQL

SmallRye JWT

Dev Services

Autor do desafio

Para Jayme · estudo de Quarkus

Escopo

MVP · ~1 fim de semana

Objetivo

Comparar padrões Quarkus vs Spring Boot

01 Visão geral

O **MotoLog** é um sistema onde um piloto cadastra suas motos e registra cada manutenção realizada (troca de óleo, filtro de ar, corrente, revisão de válvulas, pneus, etc.), acompanhando data, horas de motor / quilometragem e custo. A partir dos intervalos recomendados por tipo de serviço, o sistema ainda indica as **próximas manutenções previstas**.

Por que este projeto: o domínio é pequeno o suficiente para caber em um fim de semana, mas obriga você a exercitar tudo que importa em uma API real — autenticação com JWT, fluxo de recuperação de senha por token, relacionamentos JPA (1:N com chave estrangeira), validação de entrada, autorização por dono do recurso e regras de negócio. É também um bom pretexto para conhecer os recursos que diferenciam o Quarkus na prática: `quarkus dev` (live coding), **Dev Services** (banco subindo sozinho) e **Panache**.

Perfil do usuário: um piloto que quer manter o histórico de manutenção da(s) própria(s) moto(s) organizado e saber quando o próximo serviço está chegando.

02 Escopo do MVP

Dentro do escopo

- Cadastro de usuário
- Login com emissão de JWT
- Solicitação de redefinição de senha (gera token + "envia" e-mail)
- Redefinição de senha via token
- CRUD de motos (apenas do usuário logado)
- Registro e histórico de manutenções por moto
- Consulta de próximas manutenções previstas

bônus

Fora do escopo (não faça agora)

- Front-end (a entrega é a API; teste via Swagger/cURL)
- Upload de fotos / anexos de nota fiscal
- Perfis sociais, compartilhamento entre usuários
- Confirmação de e-mail no cadastro
- Refresh token / logout com blacklist
- Paginação avançada e filtros complexos

Dica de disciplina: se bater vontade de adicionar algo, anote como "v2". O valor do exercício está em *terminar* um sistema completo e coeso no prazo.

03 Stack e extensões Quarkus

Crie o projeto com o CLI e adicione as extensões abaixo. Cada uma tem um papel claro no sistema.

Extensão	Para quê	Equivalente no Spring
<code>quarkus-rest</code> + <code>quarkus-rest-jackson</code>	Endpoints REST (Jakarta REST) + JSON	<code>spring-boot-starter-web</code>
<code>quarkus-hibernate-orm-panache</code>	Persistência JPA simplificada	<code>spring-boot-starter-data-jpa</code>
<code>quarkus-jdbc-postgresql</code>	Driver PostgreSQL	<code>postgresql driver</code>
<code>quarkus-hibernate-validator</code>	Validação de DTOs (<code>@NotBlank</code> , <code>@Email</code>)	<code>spring-boot-starter-validation</code>
<code>quarkus-smallrye-jwt</code> + <code>-build</code>	Verificar e emitir tokens JWT	<code>spring-boot-starter-security</code> + <code>jjwt</code>
<code>quarkus-elytron-security-common</code>	Hash de senha com BCrypt (<code>BcryptUtil</code>)	<code>BCryptPasswordEncoder</code>
<code>quarkus-mailer</code>	Enviar e-mail de reset (em dev, cai no log)	<code>spring-boot-starter-mail</code>
<code>quarkus-smallrye-openapi</code> bônus	Swagger UI automático	<code>springdoc-openapi</code>

Não configure banco em dev. Tendo a extensão do PostgreSQL no classpath e nenhuma URL de banco definida, o **Dev Services** sobe um container PostgreSQL automaticamente ao rodar `quarkus dev` — inclusive para os testes. Você só define a URL real no profile `%prod`. Isso substitui o `docker-compose` que você usaria no Spring.

04 Modelo de dados

Relacionamentos

- **User 1:N Motorcycle** — um usuário tem várias motos
- **User 1:N PasswordResetToken** — um usuário pode gerar vários tokens de reset
- **Motorcycle 1:N MaintenanceRecord** — uma moto tem vários registros de manutenção
- **MaintenanceType 1:N MaintenanceRecord** — um tipo de serviço classifica vários registros

■ **PK** = chave primária · ■ **FK** = chave estrangeira

Tabela **tb_user**

Coluna	Tipo	Restrições / observações
id	BIGINT	PK, auto-incremento (IDENTITY)
name	VARCHAR(120)	NOT NULL
email	VARCHAR(180)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL — hash BCrypt, nunca a senha em texto
role	VARCHAR(20)	NOT NULL, DEFAULT 'USER'
created_at	TIMESTAMP	NOT NULL, DEFAULT now()

Tabela **tb_password_reset_token**

Coluna	Tipo	Restrições / observações
id	BIGINT	PK, auto-incremento
user_id	BIGINT	FK → tb_user(id), NOT NULL
token	VARCHAR(255)	NOT NULL, UNIQUE — um UUID aleatório
expires_at	TIMESTAMP	NOT NULL — validade de 30 minutos
used	BOOLEAN	NOT NULL, DEFAULT false — uso único
created_at	TIMESTAMP	NOT NULL, DEFAULT now()

Tabela **tb_motorcycle**

Coluna	Tipo	Restrições / observações
id	BIGINT	PK, auto-incremento
user_id	BIGINT	FK → tb_user(id), NOT NULL — dono da moto
brand	VARCHAR(60)	NOT NULL (ex.: KTM)
model	VARCHAR(80)	NOT NULL (ex.: 350 EXC-F Six Days)
model_year	INTEGER	opcional
plate	VARCHAR(10)	opcional
current_km	INTEGER	opcional — hodômetro atual
current_engine_hours	NUMERIC(7,1)	opcional — horas de motor (enduro)
created_at	TIMESTAMP	NOT NULL, DEFAULT now()

Tabela `tb_maintenance_type` (tabela de apoio / seed inicial)

Coluna	Tipo	Restrições / observações
<code>id</code>	BIGINT	PK, auto-incremento
<code>name</code>	VARCHAR(80)	NOT NULL, UNIQUE (ex.: Troca de óleo)
<code>interval_km</code>	INTEGER	opcional — intervalo recomendado em km
<code>interval_engine_hours</code>	NUMERIC(7,1)	opcional — intervalo em horas de motor

Seed sugerido (via `import.sql`): *Troca de óleo, Filtro de ar, Filtro de óleo, Corrente e coroa, Pastilhas de freio, Pneu, Revisão de válvulas, Fluido de freio.*

Tabela `tb_maintenance_record`

Coluna	Tipo	Restrições / observações
<code>id</code>	BIGINT	PK, auto-incremento
<code>motorcycle_id</code>	BIGINT	FK → <code>tb_motorcycle(id)</code> , NOT NULL
<code>maintenance_type_id</code>	BIGINT	FK → <code>tb_maintenance_type(id)</code> , NOT NULL
<code>service_date</code>	DATE	NOT NULL — não pode ser futura
<code>odometer_km</code>	INTEGER	opcional — km no momento do serviço
<code>engine_hours</code>	NUMERIC(7,1)	opcional — horas no momento do serviço
<code>cost</code>	NUMERIC(10,2)	opcional — custo do serviço
<code>notes</code>	TEXT	opcional — peças usadas, oficina, etc.
<code>created_at</code>	TIMESTAMP	NOT NULL, DEFAULT <code>now()</code>

05 Casos de uso

UC01 · Cadastrar usuário

Ator: Visitante

Pré-condição: E-mail ainda não cadastrado

Fluxo principal: informa nome, e-mail e senha → sistema valida → gera hash BCrypt → persiste → retorna 201.

Exceções: e-mail já existe → 409; dados inválidos → 422.

UC02 · Autenticar (login)

Ator: Usuário cadastrado

Fluxo principal: informa e-mail e senha → sistema confere hash → emite JWT com `sub`, `groups` (role) e expiração → retorna o token.

Exceções: credenciais inválidas → 401.

UC03 · Solicitar redefinição de senha

Ator: Usuário

Fluxo principal: informa o e-mail → sistema gera token (UUID) com validade de 30 min → envia link por e-mail (em dev, cai no log) → retorna 200.

Regra de segurança: responda 200 mesmo se o e-mail não existir, para não revelar quais e-mails estão cadastrados.

UC04 · Redefinir senha

Ator: Usuário

Pré-condição: possui token válido, não usado e não expirado

Fluxo principal: envia token + nova senha → sistema valida token → atualiza hash → marca token como usado → 200.

Exceções: token inválido/expirado/usado → 400.

UC05 · Cadastrar moto

Ator: Usuário autenticado

Fluxo principal: envia marca, modelo, ano (JWT identifica o dono) → sistema persiste vinculando ao usuário → 201.

UC06 / UC07 · Listar, atualizar e excluir motos

Ator: Usuário autenticado

Fluxo principal: lista/edita/exclui apenas as motos do próprio usuário.

Regra de autorização: se a moto não pertence ao usuário do token → 403/404.

UC08 · Registrar manutenção

Ator: Usuário autenticado (dono da moto)

Fluxo principal: escolhe a moto e o tipo de serviço, informa data, km/horas e custo → sistema valida (data não futura, moto é do usuário) → persiste → 201.

UC09 · Consultar histórico de manutenção

Ator: Usuário autenticado (dono da moto)

Fluxo principal: lista os registros da moto, ordenados por data (mais recente primeiro).

UC10 · Próximas manutenções previstas **bônus**

Ator: Usuário autenticado (dono da moto)

Fluxo principal: para cada tipo com intervalo definido, calcula (última manutenção + intervalo) e compara com km/horas atuais da moto, indicando o que está próximo ou vencido.

06 Mapa de endpoints

Endpoint	Auth?	Descrição
POST /auth/register	não	UC01 — cadastro
POST /auth/login	não	UC02 — retorna JWT
POST /auth/forgot-password	não	UC03 — gera token de reset
POST /auth/reset-password	não	UC04 — redefine com token
GET /motorcycles	JWT	UC06 — minhas motos
POST /motorcycles	JWT	UC05 — cadastrar moto
PUT /motorcycles/{id}	JWT	UC07 — atualizar
DELETE /motorcycles/{id}	JWT	UC07 — excluir
GET /motorcycles/{id}/maintenances	JWT	UC09 — histórico
POST /motorcycles/{id}/maintenances	JWT	UC08 — registrar serviço
GET /motorcycles/{id}/maintenances/upcoming	JWT	UC10 — previstas bônus
GET /maintenance-types	JWT	lista tipos de serviço

07 Regras de negócio e validações

- E-mail é único no sistema.
- Senha com no mínimo 8 caracteres; armazenada apenas como hash **BCrypt**.
- Token de reset expira em **30 minutos** e é de **uso único**.
- `/auth/forgot-password` sempre responde 200, mesmo para e-mail inexistente (não vaziar cadastro).
- Um usuário só enxerga e altera **as próprias motos e manutenções** (autorização por dono).
- `service_date` não pode ser uma data futura.
- Todos os endpoints, exceto os de `/auth`, exigem JWT válido.

08 Plano de fim de semana

Sábado de manhã — Fundação ~3h

`quarkus create app`, adicionar extensões, subir com `quarkus dev` e ver o Dev Services levantar o Postgres. Modelar as 5 entidades com Panache e conferir as tabelas criadas. Popular `tb_maintenance_type` via `import.sql`.

Sábado à tarde — Autenticação ~4h

UC01 (register com `BcryptUtil.bcryptHash`) e UC02 (login emitindo JWT com `quarkus-smallrye-jwt-build`). Gerar o par de chaves RSA, configurar o emissor e proteger um endpoint de teste com `@RolesAllowed`.

Domingo de manhã — Reset de senha ~3h

UC03 e UC04: gerar token, persistir com validade, "enviar" e-mail com `quarkus-mailer` (em dev, o link aparece no log), validar e trocar a senha. Testar o caminho feliz e os de erro (token expirado/usado).

Domingo à tarde — Domínio + fechamento ~4h

CRUD de motos (UC05–07) e manutenções (UC08–09) com a regra de autorização por dono lida do JWT. UC10 se sobrar tempo. Testar tudo pelo Swagger UI e conferir a Definition of Done.

09 Quarkus vs Spring Boot — o que observar

Enquanto constrói, repare em como cada conceito que você já conhece do Spring aparece aqui. Essa é a parte que vira ouro em entrevista.

Conceito	Spring Boot	Quarkus (neste projeto)
Injeção de dependência	<code>@Autowired</code> / <code>@Service</code>	<code>@Inject</code> / <code>@ApplicationScoped</code> (CDI)
Controller REST	<code>@RestController</code> + <code>@GetMapping</code>	<code>@Path</code> + <code>@GET</code> (Jakarta REST)
Persistência	Spring Data <code>JpaRepository</code>	Panache (active record ou repository)
Segurança / JWT	Spring Security + filtros	SmallRye JWT + <code>@RolesAllowed</code>
Configuração	<code>application.yml</code> + profiles	<code>application.properties</code> com <code>%dev</code> / <code>%prod</code>
Banco em dev	docker-compose / instância manual	Dev Services (sobe sozinho)
Modo dev	<code>spring-boot:run</code> + devtools	<code>quarkus dev</code> (live reload por request)
Startup / memória	segundos / footprint maior	milissegundos / footprint menor

10 Definition of Done e extras

Pronto quando... checklist

- Consigo registrar, logar e receber um JWT válido.
- Consigo solicitar e concluir a redefinição de senha.
- Endpoints protegidos recusam requisições sem token (401).
- CRUD de moto e registro de manutenção funcionando.
- Só enxergo as minhas motos — a de outro usuário retorna 403/404.
- Data de manutenção futura é rejeitada com 422.

Se sobrar tempo (v2)

- Testes com `@QuarkusTest` + RestAssured.
- Swagger UI com `quarkus-smallrye-openapi`.
- Compilar em **native** e medir o tempo de boot — e relacionar com o problema de cold start que você já viu no Cloud Run.
- Dockerfile e deploy num container.
- Endpoint de próximas manutenções (UC10) com o cálculo por intervalo.

Comando para começar: `quarkus create app br.com.motolog:motolog` → entre na pasta → `quarkus ext add rest-jackson hibernate-orm-panache jdbc-postgresql hibernate-validator smallrye-jwt smallrye-jwt-build elytron-security-common mailer` → `quarkus dev`.